

AutoKGS: 跨学科知识图谱智能体

两人分工：

徐志诚（50%）：

- 设计 Planner/Miner/Validator/Relation 等 6 类原子智能体
- 构建线性与并发混合的动态工作流，模拟人类专家的“规划-检索-验证-关联”研究范式
- 使用RAG技术以降低大模型幻觉
- 实现 "Local First, Online Fallback" 策略：优先检索本地 ChromaDB 向量库，置信度不足时自动触发 Kimi 联网搜索 ArXiv/Wikipedia，兼顾准确性与时效性
- 封装 ECNU/Moonshot 双模型路由，实现 Prompt 工程调优
- 开发数据ETL工具链，实现非结构化教科书的自动化向量入库与数据库迁移
- 实现 Worker 任务引擎，打通“Redis 消息 -> Agent 推理 -> Neo4j 图谱构建”的全链路数据流

周可轩（50%）：

架构与前端：

- 设计系统整体架构：前端 / 后端 / Worker / Redis / Neo4j / ChromaDB 的云原生拆分与协作流程
- 完成 Docker Compose 编排与环境配置，保证多服务一键启动
- 定义任务流转协议（Redis 任务队列与状态更新），保证前后端进度可视化
- 构建图谱可视化界面（React + ECharts），支持图谱 / 思维导图双视图
- 设计主干图 / 探索层切换逻辑，降低认知负担，提升展示清晰度
- 完成证据与可信度展示（边级证据卡片、节点置信度）
- 增加历史记录功能（本地搜索历史一键复用）与导出功能

1. 项目背景与目标

痛点场景：

在现代教育与科研中，知识碎片化现象严重。学生难以洞察不同学科间（如神经科学与深度学习、热力学与信息论）的内在关联，导致知识体系割裂。

核心目标：

构建一个基于多智能体（Multi-Agent）协作的系统，自动挖掘跨领域概念的桥梁，并构建可视化的知识图谱，帮助用户发现“远亲概念”的逻辑联系。

功能清单：

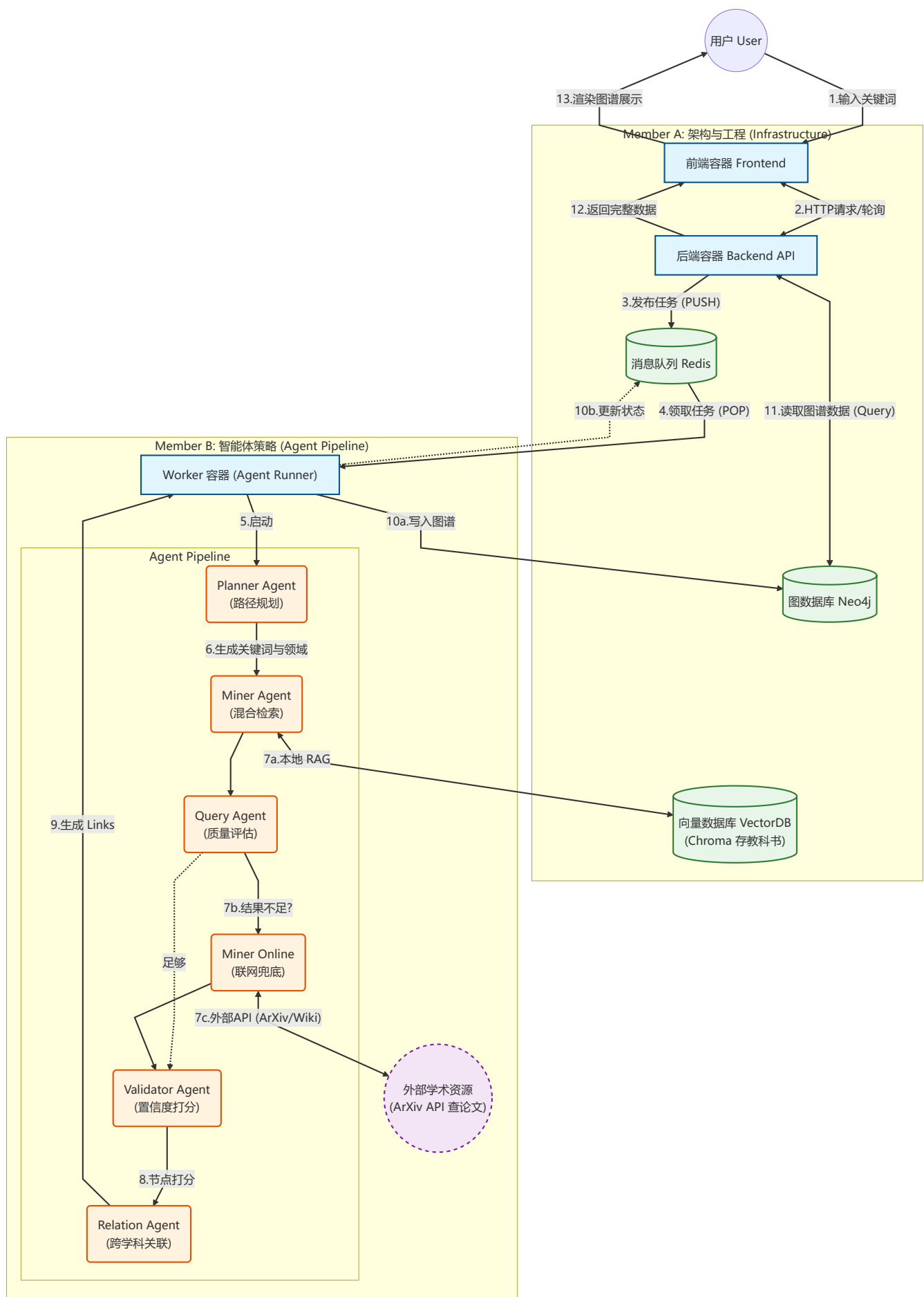
- 深度关联挖掘：**利用智能体在不同学科领域（数学、物理、社会学等）自动寻找相关概念。
- 自动化图谱构建：**从非结构化文本中提取实体及其关系，生成标准化的图结构数据。
- 混合检索增强 (RAG + Online)：**结合本地教科书知识库 (RAG) 与在线学术资源 (ArXiv/Wikipedia)，确保知识的广度与深度。
- 交互式可视化：**提供 Web 端动态交互界面，支持力导向图的拖拽、缩放与点击跳转。

- **多视图切换**: 支持知识图谱 (Knowledge Graph) 与 思维导图 (Mind Map) 两种视图模式, 满足不同场景下的知识探索需求。
- **幻觉校验机制**: 引入 Query Agent(质量评估智能体) 和 Validator Agent(置信度打分智能体) 对生成内容评估是否需要联网增强和是否达到可以展示的置信度标准, 拒绝大模型的幻觉影响。
- **证据与可信度呈现**: 关系边展示 desc/citation, 并以 confidence 强化高可信节点主干图。
- **筛选与导出**: 支持主干图(设置置信度阈值)和探索层(展示全部节点)的切换, 支持导出当前图谱 JSON 文件。
- **知识库导入**: 提供工具自动扫描并导入 EPUB/MOBI 格式教科书到向量数据库。

2. 系统架构 (System Architecture)

本系统采用 微服务架构 与 云原生 (Cloud-Native) 部署方案, 实现了计算、存储与表现层的完全解耦。

2.1 核心流程



2.2 技术栈概览

后端与架构 (Backend & Infrastructure)

模块	技术选型	技术方案说明
API 网关	FastAPI (Python)	采用高性能 ASGI 框架，利用 <code>async/await</code> 特性处理高并发长连接，自动生成 OpenAPI 规范文档。
消息队列	Redis	引入内存级中间件实现异步解耦，作为任务缓冲池，确保高并发下系统的稳定性。
容器编排	Docker Compose	采用云原生部署方案，通过声明式配置管理多容器生命周期，确保环境一致性。

前端与可视化 (Frontend & Visualization)

模块	技术选型	技术方案说明
前端框架	React + Vite	基于组件化架构构建单页应用 (SPA)，实现数据与视图分离，提供流畅的用户交互体验。
图谱渲染	Apache ECharts	使用高性能可视化引擎渲染大规模力导向图，支持节点的高亮、折叠与动态交互。
UI 组件库	Ant Design + Framer Motion	采用企业级 UI 设计语言与 Framer Motion 动画库，构建具备磨砂玻璃质感 (Glassmorphism) 与流畅动效的沉浸式界面。

智能体 (Agent System)

模块	技术选型	技术方案说明
Agent 编排	Custom Workflow	采用线性工作流 + 并发执行 (Planner → Miner/MinerOnline → Validator → Relation) 架构，支持多线程并发检索。
大模型支持	ECNU-LLM / Kimi	支持接入 ECNU 校内 LLM 或 Moonshot AI (Kimi) API 进行语义理解与生成。
图数据库	Neo4j	使用原生图数据库存储知识实体及其拓扑结构，利用 Cypher 语言高效执行多跳查询。
RAG 检索	ChromaDB	部署本地向量数据库支持 RAG (检索增强生成)，对教科书进行高维向量索引，弥补模型知识盲区。

2.3 核心技术实现细节

1) 架构设计与云原生组件

- **系统架构图**：见上文 2.1（Frontend / Backend / Worker / Redis / Neo4j / ChromaDB）。
- **云原生组件**：Docker Compose 统一编排，Redis 作为任务队列，Neo4j 持久化图谱，ChromaDB 作为向量检索服务，Worker 执行 LLM Agent 工具链。
- **微服务拆分**：前端展示、后端 API、智能体 Worker 各自独立部署，便于扩展与替换。
- **可选扩展**：可进一步迁移到 K8s（部署多副本 Worker、分离向量数据库存储）。

2) LLM Agent 工具链与 Prompt 设计

工具链组件：

- `agents/`: Planner / Miner / Query / MinerOnline / Validator / Relation
- `tools/`: `rag_retriever.py`, `search_arxiv.py`, `search_wikipedia.py`, `ecnu_llm.py`, `kimi_llm.py`

关键 Prompt 模板：

- `planner.txt`：抽取用户输入中的核心关键词与学科域

你是一个精准的意图识别助手。你的任务是从用户的查询中提取关键词（**Keywords**）和明确指定的学科范围（**Subjects**）。

请严格遵守以下规则：

1. 关键词提取：

- 提取最核心的1-4个关键词。
- 优先使用中文，除非英文术语（如"Transformer"）比中文翻译（如"变形器"）更通用且无歧义。
- 将缩写还原为全称。

2. 学科提取（关键）：

- 绝对禁止推理或猜测学科。只有当用户在查询中显式提到了某个学科名称（或其明显别名）时，才将其加入`subjects`列表。
- 例如：
 - 用户输入 "熵在信息论中的定义" -> `subjects: ["Language_and_Symbolic_Systems"]`（假设信息论归为此类）
 - 用户输入 "熵" -> `subjects: []`（空列表，不要因为熵属于物理就填物理）
- 输出的学科必须严格匹配此列表：{`subject_list`}。如果不匹配，请忽略。
- **再次强调：如果用户没有明确指定学科，`subjects` 必须为空列表 `[]`。

输出格式：

以 JSON 格式输出结果，结构如下：

```
```json
{
 "keywords": ["keyword1", "keyword2" ...],
 "subjects": ["学科1", "学科2" ...],
}
```

请分析给出的文本，并直接输出json，无需额外的解释说明。

- `query.txt`：判断本地结果是否需要联网兜底

你是一个严谨的知识质量审核员。你的任务是判断本地知识库（**RAG**）检索到的信息是否足以解释给定的关键词。

请基于以下标准进行判断：

- 信息完整性：检索到的内容是否提供了关于该关键词的清晰定义、核心概念或背景描述？
- 相关性：内容是否与该关键词所在的学科领域紧密相关？
- 可读性：提取的信息是否是连贯的语句，而非破碎的片段？

如果当前检索结果内容为空，或者包含“未找到”、“不知道”、“缺乏信息”等含义，或者内容完全不相关、极其简略无法构成有效知识点，请回答 **yes**（代表需要进行联网搜索）。

如果内容质量尚可，能够提供基本的解释，请回答 **no**（代表无需联网，保留本地结果）。

只返回 "yes" 或 "no"，不要包含任何解释或其他标点符号。

- `miner.txt`：从 RAG chunk 汇总实体节点

你是{sub\_category}领域的专家，你将得到一个json文件，为：

```
{
 "keyword": "{keyword}",
 "chunk": [chunk1, chunk2, ...],
}
```

你必须严格依照chunk中的内容，整合chunk中所有该{sub\_category}领域关于{keyword}的知识，并给出解释。

输出格式：

以 JSON 格式输出结果，结构如下：

```
```json
{
  "label": "{keyword}",
  "group": "{subject_name}--{sub_category}",
  "size": 参考的权重数，参考了多少个chunk，
  "source_type": "textbook",
  "source": "来源的书籍的名称，选取你参考的最多的1本书",
  "url": "",
  "info": "关键词解释",
}
```
```

请分析给出的文本，并直接输出json，无需额外的解释说明。

- `miner_online.txt`：从 ArXiv/Wikipedia 汇总实体节点

你是{subject\_name}领域的专家，你需要围绕{keyword}，做：

- 调用arxiv的api查找{subject\_name}领域的论文，
- 从wikipedia上直接搜索{subject\_name}中的{keyword}定义

你必须严格依照联网搜索中的内容，整合所有该{subject\_name}领域关于{keyword}的知识，并给出解释。

输出格式：

以 JSON 格式输出结果，结构如下：

```
```json
{
  "label": "{keyword}",
  "group": "subject_name",
  "size": 参考了多少资料，资料参考的权重数，参考了多少个网站，
  "source_type": "paper"或者"wiki"取决于你用哪份资料用的更多，
}
```

```
"source": "网页标题或者论文标题,选取参考的最多的",
"url": "来源的源网页, 选取参考的最多的",
"info": "关键词解释",
}
...
```

请分析给出的文本，并直接输出json，无需额外的解释说明。

- `relation.txt`：跨学科定义对齐，生成关系边

你是{subject_name1}与{subject_name2}交叉领域的专家，你将得到两段定义相同keyword的不同领域的两句话，格式为：

```
{
  "keyword": "关键词",
  "subject_name1": "定义1",
  "subject_name2": "定义2",
}
```

注意你要严格遵循以下准则：

- 若两者没有联系，则返回空json文件，即{}

输出内容：你需要输出json格式文件：

```
```json
{
 "source": "关键词",
 "target": "两个领域对该关键词的交叉之处，使用10字以内概括",
 "relation": "两个关系的联系，用一个全大写的英文短语概括，例如定义1 USES 定义2，就写USES，定义1是主
语",
 "desc": "为什么有此联系的原因",
}
...
```

只用输出json文件，不要有其他任何的输出

- `val.txt`：输出置信度评分（0-1）

你是{subject\_name}领域下{sub\_category}方向的专家，你需要围绕{keyword}与你的领域，给它的定义评分，最后的分数在0到1之间，请你直接给出评分，不要给出任何多余文字

## Agent 设计：

### 1. 混合检索策略 (Hybrid Retrieval)

系统采用 "Local First, Online Fallback" 的混合检索策略，确保知识的准确性与时效性：

- **Level 1: Local RAG (优先)：**
  - 利用 `ChromaDB` 存储本地教科书的高维向量索引。
  - 使用 `ecnu-embedding-small` 模型将用户查询与教科书内容进行语义匹配，提取高置信度的学术知识。
  - 优势：知识来源权威、响应速度快、无幻觉。
- **Level 2: Online Search (兜底)：**

- 当本地知识库无法满足需求时（如前沿概念或缺失学科），自动触发联网检索。
- 调用 `MinerOnlineAgent` 访问外部 API (ArXiv/Wikipedia) 获取最新论文摘要和定义。
- 优势：覆盖面广、时效性强。

## 2. 多智能体协作 (Multi-Agent Collaboration)

系统通过多个专注于特定任务的智能体协同工作，模拟人类专家的研究流程：

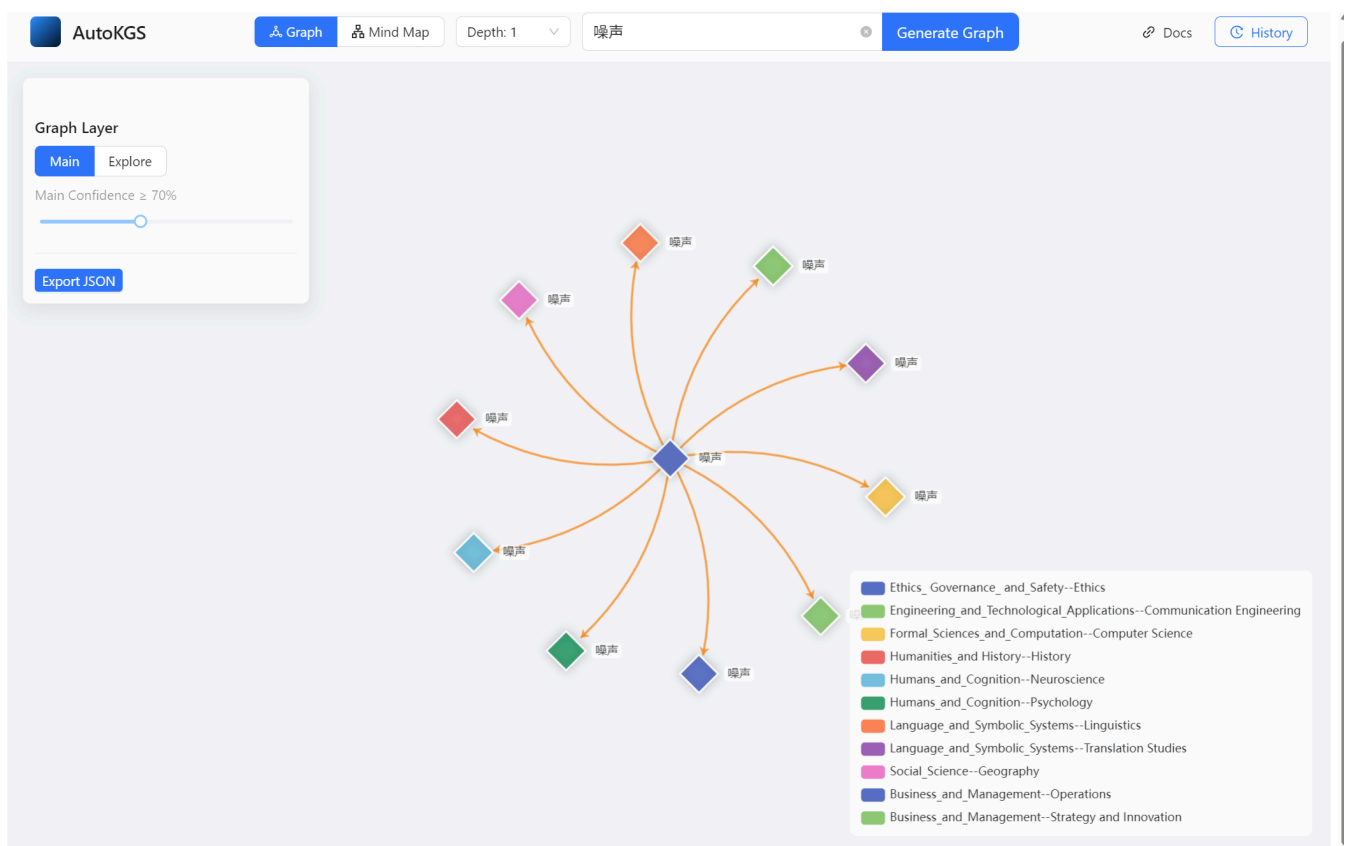
- **Planner Agent**: 基于用户输入（如“熵”），动态规划需要探索的学科领域（如物理学、信息论、统计学），避免盲目搜索。
- **Miner Agent**: 执行具体的挖掘任务，从 RAG 或网络中提取实体（Entities）和概念定义。
- **Validator Agent**: 扮演“审稿人”角色，对提取的实体进行置信度打分（0-1），过滤低质量或不相关的内容。
- **Relation Agent**: 专门负责发现实体间的跨学科关联（如“熵”在热力学与信息论中的数学同构性），构建图谱的边（Links）。

## 3) 前端设计思路与功能

- **主干图 / 探索层**: 用 `confidence` 构建 Main 视图，Explore 展示全量关系。
- **证据展示**: 顶点和边级卡片均展示详细内容，强调可解释性。
- **多视图**: 图谱与思维导图双视图切换，满足不同学习场景。
- **历史记录**: 本地记录搜索历史，一键复用查询。
- **导出**: 导出当前图谱 JSON，便于复现与汇报。
- **可选深度**: 适应读者的需求，探索不同程度的关系

可视化展示：

图谱全貌展示：





顶点卡片展示：

噪声

**[object Object]**

Type: TEXTBOOK

Source: 工程伦理 -- 王晓敏,王浩程 编著.epub

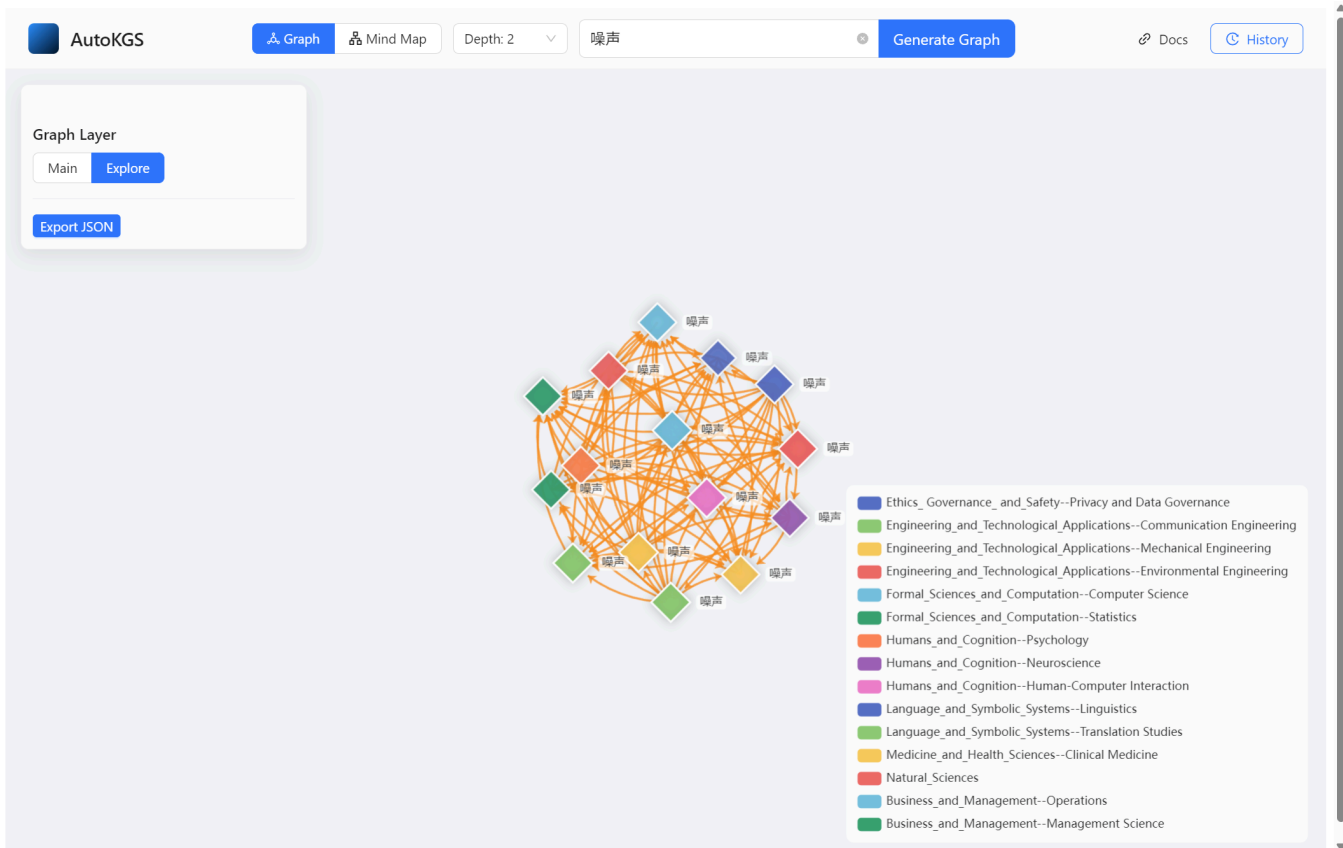
Confidence: 85%

在工程伦理语境下，'噪声'虽未直接定义，但其隐含于对环境风险因素的讨论中。噪声作为客观环境因素之一，可视为影响工人健康与安全的重要污染源。尤其在小规模生产企业中，粉尘弥漫、气味刺鼻等恶劣工作环境常伴随噪声污染，加剧工人的健康风险和伤害风险。从伦理角度看，企业为节省成本忽视生

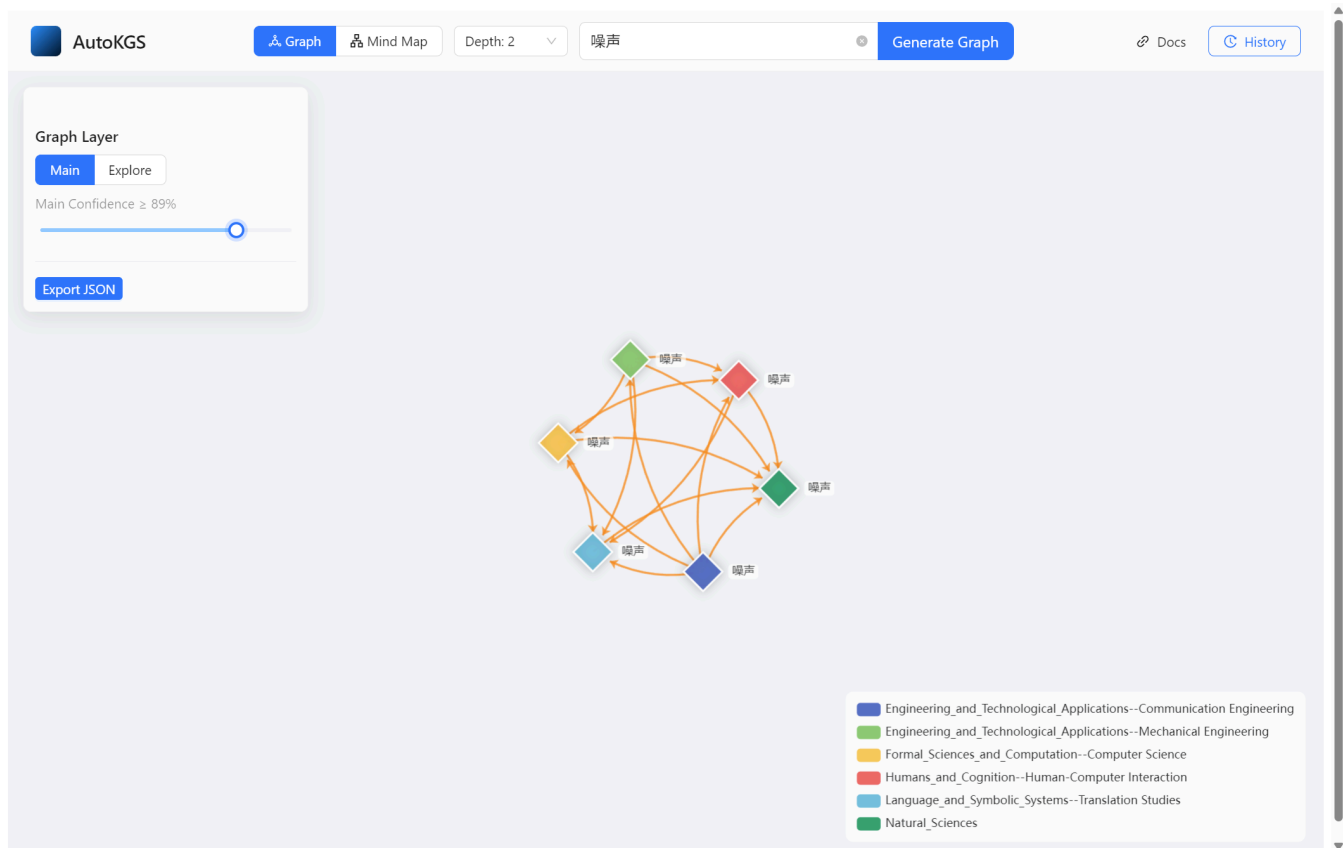
边卡片展示：



调整深度后的Explore图展示：



调整置信度后的Main图展示：



导出JSON文件展示：

```
{} autokgs_噪声.json X
C:\Users\zhich> Downloads > {} autokgs_噪声.json > {} nodes > {} 1
1 {
2 "nodes": [
3 {
4 "confidence": 0.95,
5 "source": "通信原理 -- 黄根岭 主编.epub",
6 "id": "a1f01720-a286-4408-a27d-f7f4c17475e3",
7 "label": "噪声",
8 "task_ids": [
9 "e7d70a90-f02d-4254-baab-613023da0c76"
10],
11 "url": "",
12 "group": "Engineering_and_Technological_Applications--Communication Engineering",
13 "info": "在通信系统中，噪声是影响信号传输质量的重要因素。常见的噪声类型包括白噪声、高斯噪声、高斯白噪声和窄带高斯噪声。白噪声的功率谱密度在整个频域内是均匀的。高斯噪声的幅度服从高斯分布。高斯白噪声的功率谱密度在整个频域内是均匀的。窄带高斯噪声的功率谱密度集中在某个窄带内。",
14 },
15 {
16 "confidence": 0.95,
17 "source": "机械设计同步辅导及习题全解",
18 "id": "0462d676-2b7a-4d56-8ddc-560f07e05f55",
19 "label": "噪声",
20 "task_ids": [
21 "e7d70a90-f02d-4254-baab-613023da0c76"
22],
23 "url": "",
24 "group": "Engineering_and_Technological_Applications--Mechanical Engineering",
25 "info": "在机械工程领域，噪声通常指机械设备运行过程中产生的不需要的、干扰性的声波振动。它可能源于齿轮啮合、轴承摩擦、空气动力效应或结构共振等。",
26 },
27 {
28 "confidence": 0.95,
29 "source": "计算机网络(第7版) ('十二五'普通高等教育本科国家级规划教材) -- 谢希仁.epub",
30 "id": "07b67127-6d6b-4ffe-af88-1970a3beb491",
31 "label": "噪声",
32 "task_ids": [
33 "e7d70a90-f02d-4254-baab-613023da0c76"
34],
35 "url": "",
36 "group": "Formal_Sciences_and_Computation--Computer Science",
37 "info": "噪声存在于所有电子设备和通信信道中，是随机产生的，其瞬时值有时会很大，导致接收端对码元判决错误（如1误判为0或0误判为1）。噪声的影响程度与信噪比有关。",
38 }
39],
40 }
```

### 3. 快速开始 (Quick Start)

本项目基于 Docker 构建，可实现一键部署。

#### 前置要求

- Docker Desktop 或 Docker Engine
- Docker Compose

#### 启动步骤

##### 1. 克隆项目

```
git clone <repository_url>
cd AutoKGS
```

##### 2. 配置环境变量

由于 `.env` 文件包含敏感信息，通常不包含在代码仓库中。你需要手动创建 `.env` 文件。

在项目根目录下创建 `.env` 文件，并填入以下内容：

```
--- Neo4j 数据库配置 ---
NEO4J_URI=bolt://neo4j:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=password # 请修改为你想要的密码

--- Redis 配置 ---
REDIS_HOST=redis
REDIS_PORT=6379
```

```
--- ChromaDB 配置 ---
CHROMA_SERVER_HOST=chromadb
CHROMA_SERVER_PORT=8000

--- LLM 配置 ---

1. ECNU LLM (推荐, 用于本地 RAG 和逻辑处理)
LLM_API_KEY=your_ecnu_api_key
LLM_API_BASE=https://api.ecnu.edu.cn/v1
EMBEDDING_MODEL=ecnu-embedding-small

2. Kimi (Moonshot AI) (用于联网检索兜底)
KIMI_API_KEY=sk-xxxxxxx
KIMI_API_BASE=https://api.moonshot.cn/v1
```

### 3. 启动服务

```
docker-compose up --build -d
```

### 4. 访问服务

- 前端界面: <http://localhost:5173>
- 后端 API: <http://localhost:8000/docs>
- Neo4j 控制台: <http://localhost:7474> (账号: neo4j / 密码: 你在.env中设置的密码)
- ChromaDB: <http://localhost:8001>

## 数据导入 (RAG 增强)

为了让 AI 拥有更专业的学科知识, 你可以导入本地教科书数据。

#### 数据资源:

我们提供了一份基础的学科分类教科书数据集:

- 下载链接: [https://pan.baidu.com/s/1aOxGAS\\_UwNEWgix-l9Y4wA?pwd=6ju3](https://pan.baidu.com/s/1aOxGAS_UwNEWgix-l9Y4wA?pwd=6ju3)
- 提取码: 6ju3

还提供了经处理后的教科书的向量集:

- 下载链接: <https://pan.baidu.com/s/1wlaPRWgOmV66hAy9oRtgBQ?pwd=wwhy>
- 提取码: wwhy

#### 导入方式 (任选其一):

**方式 A: 扫描教科书文件 (链接可能因版权问题而失效, 此时需要切换方式B或者联系我们获取)**

1. 下载并解压 `data.7z`。
2. 将解压得到的 `data` 文件夹 (包含 `.epub` / `.mobi`) 放在项目根目录的 `/data`。  
(注: `/data` 已映射到容器内 `/app/data`)
3. 执行导入脚本:

```
docker compose exec worker bash
python -m tools.ingest_books
```

脚本会递归扫描 `/app/data` 并写入 `ChromaDB`。

## 方式 B：导入 ChromaDump（快速恢复）

1. 将 `chroma_dump.json` 放到项目根目录的 `/data`（容器内路径为 `/app/data/chroma_dump.json`）。
2. 执行导入脚本：

```
docker compose exec worker bash
python -m tools.import_chromadb
```

脚本会按 `collection` 批量 `upsert` 到 `ChromaDB`。

## 4. 项目结构

```
AutoKGS/
├─ docker-compose.yml # [核心] 容器编排配置
├─ README.md # 项目文档
├─ .env # 环境变量
├─ .gitignore
├─
├─ frontend/ # [服务1] 前端容器 (React + Vite)
│ ├─ Dockerfile
│ ├─ src/
│ │ ├─ components/
│ │ ├─ api/
│ │ ├─ useGraphOption.js # 图谱配置
│ │ ├─ useMindMapOption.js # 思维导图配置
│ │ └─ App.jsx
│ └─ ...
├─
├─ backend/ # [服务2] 后端 API 容器 (FastAPI)
│ ├─ Dockerfile
│ ├─ main.py # FastAPI 入口
│ ├─ routers/ # 路由定义 (Task, Graph)
│ └─ database/ # 数据库连接工具
├─
├─ agent_engine/ # [服务3] 智能体 worker 容器 (Python)
│ ├─ Dockerfile
│ ├─ worker_main.py # worker 主程序 (监听 Redis)
│ ├─ workflow.py # Agent 工作流编排
│ ├─ config.py
│ ├─
│ └─ agents/ # 智能体实现
│ ├─ planner.py # 规划 Agent
│ ├─ miner.py # 挖掘 Agent (RAG)
│ ├─ miner_online.py # 联网 Agent
│ └─ query.py # 质检 Agent
```

```

| | | └─ validator.py # 验证 Agent
| | | └─ relation.py # 关系 Agent
| | |
| | └─ tools/ # 工具链
| | | └─ ingest_books.py # 图书导入工具
| | | └─ ecnu_llm.py # LLM 封装
| | | └─ search_arxiv.py # ArXiv 工具
| | | └─ rag_retriever.py # RAG 检索器
| | |
| | └─ prompt/ # Prompt 模板
| |
└─ neo4j_data/ # Neo4j 数据持久化
└─ chroma_data/ # ChromaDB 数据持久化
└─ redis_data/ # Redis 数据持久化

```

## 5. 数据交互格式

### 任务状态 (Redis)

后端轮询 `task:{uuid}` 获取实时进度:

```

{
 "status": "PROCESSING",
 "step_index": 2,
 "progress": 45,
 "message": "正在并行检索 (1/4)..."
}

```

### 图数据结构 (Neo4j -> Frontend)

```

{
 "nodes": [
 {
 "id": "UUID",
 "label": "熵",
 "group": "Natural_Sciences",
 "source_type": "textbook",
 "confidence": 0.98,
 "info": "..."
 }
],
 "links": [
 {
 "source": "UUID_A",
 "target": "UUID_B",
 "relation": "RELATED_TO",
 "citation": "Source A; Source B"
 }
]
}

```

# Redis格式:

## 任务发布:

```
{
 "task_id": "550e8400-e29b-41d4...", //UUID, 唯一凭证
 "keyword": "熵", // 用户输入
 "params": { // (可选) 额外参数
 "depth": 3, // 搜索深度
 "language": "zh"
 },
 "created_at": 1709876543
}
```

## 任务状态格式:

存入 Redis Key-Value (Key: task:550e8400...) 后端接口会不断轮询读取这个 Key。

### 1. 状态流转定义 (Step Definitions)

前端根据 step\_index (0-4) 展示不同的加载动画步骤。后端需按照以下阶段更新 Redis:

Step	Status	Progress	Message (示例)	说明
0	PROCESSING	5	"任务已发送至 Redis 队列..."	初始状态, 等待 Worker 领取
1	PROCESSING	20	"AI Worker (Agent) 已接单..."	Worker 开始执行
2	PROCESSING	45	"正在检索 ArXiv 和教科书..."	Miner Agent 进行混合检索
3	PROCESSING	70	"正在提取实体与关系..."	LLM 阅读文本并抽取知识
4	PROCESSING	90	"图谱构建完成, 正在渲染..."	校验通过, 写入 Neo4j
5	SUCCESS	100	"Completed"	任务彻底完成, 返回结果

### 2. JSON 示例

#### 进行中 (Processing):

```
{
 "status": "PROCESSING",
 "step_index": 2,
 "progress": 45,
 "message": "正在检索 ArXiv 和教科书..."
}
```

#### 成功 (Success):



```
{
 "status": "SUCCESS",
 "step_index": 5,
 "progress": 100,
 "result_node_id": "Entropy", // 告诉后端去 Neo4j 查哪个主节点
 "message": "Completed",
 "completed_at": 1709876599
}
```

失败 (Failed):

```
{
 "status": "FAILED",
 "error": "搜索超时，请稍后重试"
}
```